MID-TERM PROJECT REPORT — CRYPTOGRAPHY

REPORT TITLE:
Design and Implementation of a QR Code-Based Digital Certificate Verification System Using RSA Digital Signatures

(Hệ thống xác minh chứng nhận bằng mã QR sử dụng chữ ký số RSA)

MEMBER LIST & STUDENT ID

No.	Full Name (Họ và tên)	Student ID (MSSV)
1	Nguyễn Khải Điệp	23BA14051
2	Doãn Trung Dũng	23BA14070
3	Nguyễn Đinh Huyền Mai	23BA14186
4	Lương Quỳnh Anh	23BA14008
5	Phạm Vũ Khánh	23BA14148
6	Đặng Thế Hiệp	23BA14103
7	Nguyễn Phương Nam	23BA14206

Course: Cryptography
Date: November 2026

TABLE OF CONTENTS

Section	Title	Main Content Page
1	Introduction (Background, Problem Statement, Objectives, Scope)	Page 1 of 3 (PDF p.3)
2	Theoretical Background & Textbook RSA Weakness Reproduction	Page 1 of 3 (PDF p.3)
3	System Design, Compact Payload Scheme & Python Implementation	Page 1 - 2 of 3 (PDF p.3-4)
4	Experimental Testing, Performance Evaluation & Conclusion	Page 2 - 3 of 3 (PDF p.4-5)
-	References (Excluded from Page Limit)	PDF p.6
Annex A	Sample Valid vs. Tampered QR Codes, JSON Payload & Public Key	PDF p.6
Annex B	Experimental Execution Log (Test Cases & Benchmark Output)	PDF p.6
Annex C	Complete Python Implementation Source Code	PDF p.7 - 9

TABLE OF FIGURES AND TABLES

Item	Description
Table 1	Roles of Cryptographic and Encoding Components (RSA-PSS, SHA-256, Keys, QR)
Figure 1	End-to-End Certificate Issuance and Verification Workflow Diagram
Listing 1	Core Python Implementation of RSA-PSS Signing, Verification, and QR Decoding
Table 2	Security and Tamper-Detection Test Cases (5 Experimental Scenarios)
Table 3	Benchmark Results Across RSA-1024, RSA-2048, and RSA-3072 (30 Runs Average)
Figure 2	Signing/Verification Latency and QR Payload Size Comparison Chart

1. Introduction

1.1 Background and Problem Statement

Academic and professional institutions increasingly issue digital certificates instead of paper documents. However, plain digital files or printed certificates containing standard QR codes are easy to edit or counterfeit. A typical QR code only encodes text or a URL; it provides no cryptographic assurance of data integrity or issuer authenticity. If an attacker modifies the holder's name or certificate ID and regenerates the QR code, a normal scanner cannot detect the manipulation. Furthermore, many verification systems rely on a centralized database or web server. If the server goes offline or the verifier has no internet connection, verification fails. This project addresses these challenges by designing and implementing an offline, stateless QR code verification prototype based on RSA-PSS digital signatures and SHA-256.

1.2 Objectives and Scope

- **Objective 1:** Standardize certificate fields (*name, cert_id, issue_date*) into a canonical JSON format and sign them using RSA-PSS with SHA-256.
- **Objective 2:** Reproduce the mathematical weakness of unpadded (Textbook) RSA to demonstrate why probabilistic padding (RSA-PSS) is essential.
- **Objective 3:** Package certificate data, Base64 signature, and key ID (*kid*) into a QR code, and compare standard JSON packaging against a compressed binary scheme.
- **Objective 4:** Decode QR images using OpenCV and verify the signature against a trusted public key, accurately detecting any tampering.

Scope: The system runs as a local standalone prototype built in Python. It requires no user accounts, database, or backend server. The verifier is assumed to possess the trusted public key of the issuing authority.

2. Theoretical Background and Weakness Analysis

2.1 Cryptographic Components

Table 1 summarizes the cryptographic and encoding primitives used in the system.

Component	Role in the Verification System
RSA-PSS	Probabilistic signature scheme used to generate and verify digital signatures.
SHA-256	Cryptographic hash function producing a 256-bit digest inside the RSA-PSS pipeline.
Private / Public Key	Private key (d, n) signs data at the issuer; Public key (e, n) verifies data at the verifier.
QR Code	2D optical encoding for transporting the signed payload; provides no security by itself.

In RSA key generation, two large primes p and q are selected to compute the modulus $n = p * q$ and Euler's totient $phi(n) = (p - 1)(q - 1)$. With public exponent $e = 65537$, the private exponent is computed as $d = e^{-1} \bmod phi(n)$. To prevent structural attacks, RSA-PSS hashes the message M using SHA-256 to obtain $mHash = SHA256(M)$, binds it with a random salt, and applies a Mask Generation Function (MGF1) before modular exponentiation.

2.2 Reproducing Textbook RSA Weakness (Multiplicative Forgery)

To justify using RSA-PSS over raw RSA, we reproduced the multiplicative homomorphism weakness of Textbook RSA in Python. In Textbook RSA, signing a message m directly computes $s = m^d \bmod n$, and verification checks whether $m == s^e \bmod n$. Because modular exponentiation is multiplicative, an attacker who observes two valid message-signature pairs $(m1, s1)$ and $(m2, s2)$ can forge a valid signature $s3$ for a new message $m3 = (m1 * m2) \bmod n$ without knowing the private key d:

$$s3 = (s1 * s2) \bmod n = (m1^d * m2^d) \bmod n = (m1 * m2)^d \bmod n$$

In our experimental script using a 2048-bit RSA key with $m1 = 105$ and $m2 = 200$, we computed $s1 = pow(105, d, n)$ and $s2 = pow(200, d, n)$. Without using d, the attacker multiplies $s3 = (s1 * s2) \% n$ for $m3 = 21000$. Checking $pow(s3, e, n) == 21000$ returns **True** (forgery succeeds). RSA-PSS eliminates this vulnerability in two ways: (1) SHA-256 destroys algebraic structure since $SHA256(m1 * m2) != SHA256(m1) * SHA256(m2)$, and (2) the random PSS salt ensures signing the same certificate twice yields completely different signatures.

3. System Design and Implementation

3.1 System Architecture and Operational Workflow

The system operates in five sequential steps divided between the Issuer and the Verifier (Figure 1).

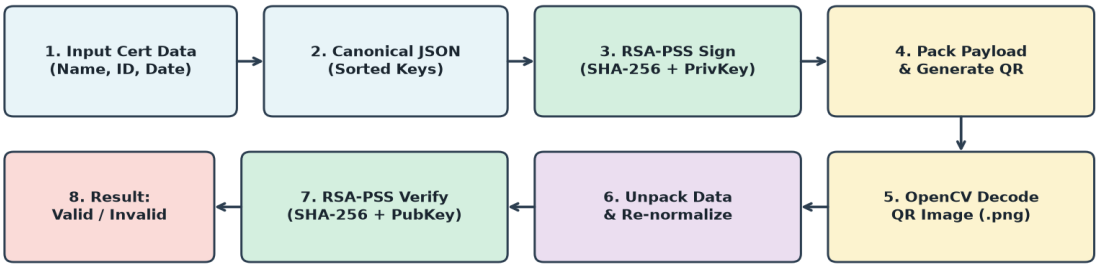


Figure 1: End-to-End Certificate Issuance and Verification Workflow.

- **Step 1 (Data Normalization):** Certificate fields (*name, cert_id, issue_date*) are serialized into canonical JSON with sorted keys (*sort_keys=True*) and compact separators so byte order is deterministic.
- **Step 2 (RSA-PSS Signing):** The canonical JSON string is hashed with SHA-256 and signed using RSA-PSS (MGF1-SHA256, maximum salt length) with the issuer's private key.
- **Step 3 (QR Packaging):** The certificate object, Base64 signature (*sig*), and key ID (*kid*) are packaged and encoded into a QR code image (.png).
- **Step 4 (Scanning & Decoding):** OpenCV (*cv2.QRCodeDetector*) decodes the QR image back into the JSON payload string.
- **Step 5 (Verification):** The extracted *data* object is re-normalized and verified against the signature using the trusted public key, returning **Valid Signature** or **Invalid Signature**.

3.2 Payload Optimization Scheme (Compact Binary + Zlib)

A 2048-bit RSA signature is 256 bytes (344 Base64 characters). Compressing the entire JSON string (including the Base64 signature) and Base64-encoding it again increases payload size because high-entropy signature bytes are incompressible. To solve this, our compact scheme (prefix *C:*) compresses only the JSON metadata using *zlib*, prepends a 2-byte length header, concatenates the raw binary signature bytes, and encodes to Base64 only once.

3.3 Core Python Implementation

The prototype is implemented in Python 3 using *cryptography*, *qrcode*, and *opencv-python*. Listing 1 shows the core functions for canonical normalization, RSA-PSS signing, verification, and QR decoding (full code in Annex C).

```
def normalize_cert_data(cert_dict):
    return json.dumps(cert_dict, ensure_ascii=False, sort_keys=True, separators=(",", ":"))

def sign_data(private_key_pem, data_str):
    priv_key = serialization.load_pem_private_key(private_key_pem.encode("utf-8"), password=None)
    sig = priv_key.sign(data_str.encode("utf-8"),
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH), hashes.SHA256())
    return base64.b64encode(sig).decode("utf-8")

def verify_signature(public_key_pem, data_str, signature_b64):
    try:
        pub_key = serialization.load_pem_public_key(public_key_pem.encode("utf-8"))
        pub_key.verify(base64.b64decode(signature_b64), data_str.encode("utf-8"),
            padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH), hashes.SHA256())
        return True
    except Exception:
        return False
```

Listing 1: Core Python Implementation for Canonical JSON, RSA-PSS Sign, and Verify.

4. Experimental Evaluation and Conclusion

4.1 Security and Tamper Detection Test Cases

Five verification scenarios were tested to evaluate tamper detection and error handling (Table 2).

Test ID	Scenario Description	Test Input Condition	Expected	Actual Output
TC-01	Valid QR and authentic signature	Authentic QR + Matching Public Key	Valid Signature	Valid Signature

TC-02	Modify holder name after signing	Name changed to 'Nguyen Van B (Fake)'	Invalid Signature	Invalid Signature
TC-03	Modify signature characters	First 10 Base64 chars altered in 'sig'	Invalid Signature	Invalid Signature
TC-04	Mismatched Public Key	Authentic QR + Wrong RSA-2048 Public Key	Invalid Signature	Invalid Signature
TC-05	Corrupted / non-QR file	Non-QR / corrupted image byte stream	Clear error msg	Cannot decode QR

4.2 Performance and QR Payload Evaluation

We benchmarked RSA-1024, RSA-2048, and RSA-3072 across 30 runs to evaluate speed and QR size (Table 3 & Figure 2).

Key Size	Sign (ms)	Verify (ms)	Signature Size	JSON Payload	Compact (C:)	QR Version
RSA-1024	3.397 ms	0.015 ms	128 B (172 chars)	316 Bytes	298 Bytes	Ver 13 (69x69)
RSA-2048	22.598 ms	0.028 ms	256 B (344 chars)	488 Bytes	470 Bytes	Ver 17 (85x85)
RSA-3072	70.811 ms	0.043 ms	384 B (512 chars)	656 Bytes	642 Bytes	Ver 20 (97x97)

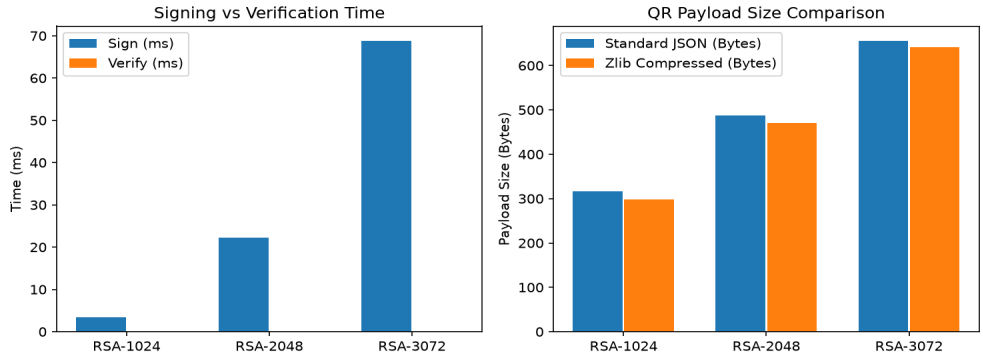


Figure 2: (Left) Signing vs. Verification Time; (Right) Standard JSON vs. Compact Payload Size.

Key Findings: (1) Verification is over 800x faster than signing for RSA-2048 (0.028 ms vs. 22.598 ms) because verification uses the short public exponent $e = 65537$, whereas signing uses a 2048-bit private exponent d. (2) Moving from RSA-2048 to RSA-3072 triples signing time and increases QR density to Version 20. Thus, RSA-2048 provides the best trade-off between 112-bit security and optical scan reliability.

4.3 Limitations and Conclusion

A valid signature confirms data integrity and private-key possession, but verifying the real-world identity of the issuer requires a trusted Public Key distribution channel (PKI), and the offline prototype does not include online revocation checking. In conclusion, this project successfully demonstrates how RSA-PSS, SHA-256, and canonical JSON packaging prevent certificate forgery and tampering in an offline QR verification workflow.

REFERENCES (Excluded from Page Limit)

1. Moriarty, K., Kaliski, B., Jonsson, J., & Rusch, A. (2016). *PKCS #1: RSA Cryptography Specifications Version 2.2* (RFC 8017). IETF.

2. National Institute of Standards and Technology (NIST). (2015). *Secure Hash Standard (SHS)* (FIPS PUB 180-4).

3. National Institute of Standards and Technology (NIST). (2023). *Digital Signature Standard (DSS)* (FIPS PUB 186-5).

4. ISO/IEC. (2015). *Information technology — QR Code bar code symbology specification* (ISO/IEC 18004:2015).

ANNEXES (Excluded from Page Limit)

Annex A: Generated QR Code Samples, Signed JSON Payload, and Sample Public Key



(a) Valid Certificate QR (TC-01)
Verification: VALID SIGNATURE



(b) Tampered Certificate QR (TC-02)
Name modified after signing -> INVALID SIGNATURE

```
Normalized Certificate Data String:
{"cert_id":"CERT-2026-001","issue_date":"2026-11-09","name":"Nguyen Van A"}

Sample Issuer RSA-2048 Public Key (PEM):
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAxvbbvsisS3wP1R+/o6Zs
u42Tl7Rk6BWbJfNwLx+LfVw3gT26JrG/AT4aOA0j92dVkJ4UZhVFSYwN1w0OpgKS
Q4DXzt2ELAFhHQTF/WOCgatSHhtkfSbwj4AQSE1+K2OejSwRQURpR+4gZwJln8fD
WSKD86knmI3d6bEnNliejLuZsm8nkO7VQORoyb0095Rix5PLXySuUO3yRdgL5Yxl
ZnRfTTKttuZFGOoovReqMXzL2Oyi/Rwc23PmPDkDuYAcZ9wEKMK3ABfCotXBVNgH
JAIq1dHHw4BMnCg74w9lxo+tfoR5/G6yarZJge1NHJRpnzLpsgn16EZKq7ooio62
pQIDAQAB
-----END PUBLIC KEY-----
```

Annex B: Experimental Test Cases & Benchmark Execution Output Log

```
=== 1. TAO CAP KHOA MAU (RSA-2048) ===
Da luu khoa mau tai thu muc sample_keys/

=== 2. CHAY 5 KICH BAN KIEM THU (TEST CASES) ===
[Case 1] QR va chu ky hop le              -> Valid Signature
[Case 2] Sua ten chung nhan sau khi ky    -> Invalid Signature
[Case 3] Thay doi chu ky                  -> Invalid Signature
[Case 4] Dung Public Key khong khop       -> Invalid Signature
[Case 5] Anh QR hong / sai dinh dang      -> Bao loi: Khong doc duoc ma QR

=== 3. CHUNG MINH DIEM YEU TEXTBOOK RSA vs RSA-PSS ===
Textbook RSA multiplicative forgery (m1=105, m2=200 -> m3=21000): Forged valid = True

=== 4. BENCHMARK & VE BIEU DO BAO CAO ===
RSA-1024: Sign=3.397ms  | Verify=0.015ms | Raw=316B (QR v13) | Zlib=298B (QR v13)
RSA-2048: Sign=22.598ms | Verify=0.028ms | Raw=488B (QR v17) | Zlib=470B (QR v17)
RSA-3072: Sign=70.811ms | Verify=0.043ms | Raw=656B (QR v20) | Zlib=642B (QR v20)
```

Annex C: Complete Python Source Code Implementation

1. Module: crypto_utils.py (RSA-PSS, SHA-256, Compact Payload & Textbook RSA Demo)

```
import json
import base64
import zlib
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.exceptions import InvalidSignature


# Tao cap khoa RSA (mac dinh 2048 bit)
def generate_rsa_keys(key_size=2048):
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=key_size
    )
    public_key = private_key.public_key()

    priv_pem = private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.NoEncryption()
    )

    pub_pem = public_key.public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo
    )

    return priv_pem.decode("utf-8"), pub_pem.decode("utf-8")


# Chuan hoa du lieu chung nhan thanh chuoi JSON co thu tu co dinh
def normalize_cert_data(cert_dict):
    return json.dumps(cert_dict, ensure_ascii=False, sort_keys=True, separators=(",", ":"))


# Ky du lieu bang RSA-PSS va SHA-256
def sign_data(private_key_pem, data_str):
    private_key = serialization.load_pem_private_key(
        private_key_pem.encode("utf-8"),
        password=None
    )

    signature = private_key.sign(
        data_str.encode("utf-8"),
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

    return base64.b64encode(signature).decode("utf-8")


# Xac minh chu ky bang public key
def verify_signature(public_key_pem, data_str, signature_b64):
    try:
        public_key = serialization.load_pem_public_key(
            public_key_pem.encode("utf-8")
        )
        sig_bytes = base64.b64decode(signature_b64)

        public_key.verify(
            sig_bytes,
            data_str.encode("utf-8"),
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH
            ),
            hashes.SHA256()
        )
        return True
    except (InvalidSignature, ValueError, Exception):
        return False
```

```
# Dong goi du lieu de dua vao QR code
# Ho tro che do rut gon (nen phan JSON + ghep chu ky nhi phan roi moi Base64 1 lan)
def pack_qr_payload(cert_dict, signature_b64, key_id="EDU-2026", use_compression=False):
    if use_compression:
        meta = json.dumps({"d": cert_dict, "k": key_id}, ensure_ascii=False, separators=(",", ":")).encode("utf-8")
        comp_meta = zlib.compress(meta, level=9)
        sig_raw = base64.b64decode(signature_b64)
        # 2 byte dau luu do dai phan meta da nen, theo sau la meta va chu ky nhi phan
        combined = len(comp_meta).to_bytes(2, "big") + comp_meta + sig_raw
        return "C:" + base64.b64encode(combined).decode("utf-8")

    payload = {
        "data": cert_dict,
        "sig": signature_b64,
        "kid": key_id
    }
    return json.dumps(payload, indent=2, ensure_ascii=False)


# Giai ma du lieu doc duoc tu QR code
def unpack_qr_payload(qr_text):
    if qr_text.startswith("C:"):
        raw = base64.b64decode(qr_text[2:])
        meta_len = int.from_bytes(raw[:2], "big")
        comp_meta = raw[2:2 + meta_len]
        sig_raw = raw[2 + meta_len:]
        meta_obj = json.loads(zlib.decompress(comp_meta).decode("utf-8"))
        return {
            "data": meta_obj["d"],
            "sig": base64.b64encode(sig_raw).decode("utf-8"),
            "kid": meta_obj["k"]
        }
    return json.loads(qr_text)


# Minh hoa diem yeu cua Textbook RSA (khong dung hash va padding PSS)
# Ke tan cong co the nhan 2 chu ky hop le de tao chu ky gia cho m3 = m1 * m2 mod n
def demo_textbook_rsa_forgery(private_key_pem):
    priv_key = serialization.load_pem_private_key(private_key_pem.encode("utf-8"), password=None)
    priv_nums = priv_key.private_numbers()
    pub_nums = priv_key.public_key().public_numbers()

    n = pub_nums.n
    e = pub_nums.e
    d = priv_nums.d

    # Gia su 2 thong diep dang so nguyen
    m1 = 105
    m2 = 200

    # Ky truc tiep khong qua RSA-PSS: s = m^d mod n
    s1 = pow(m1, d, n)
    s2 = pow(m2, d, n)

    # Ke tan cong khong biet d nhung tinh duoc chu ky cho m3 = m1 * m2
    m3_forged = (m1 * m2) % n
    s3_forged = (s1 * s2) % n

    # Kiem tra xem chu ky gia co vuot qua buoc verify (s^e mod n == m) hay khong
    verified_m = pow(s3_forged, e, n)
    is_forged_valid = (verified_m == m3_forged)

    return {
        "m1": m1,
        "m2": m2,
        "m3_forged": m3_forged,
        "is_forged_valid": is_forged_valid
    }
```

2. Module: qr_utils.py (QR Code Generation & OpenCV Decoding)

```
import io
import os
import qrcode
import cv2
import numpy as np

# Tao anh QR code tu chuoi du lieu
```

```
def create_qr_image(data_str, error_level="M"):
    level_map = {
        "L": qrcode.constants.ERROR_CORRECT_L,
        "M": qrcode.constants.ERROR_CORRECT_M,
        "Q": qrcode.constants.ERROR_CORRECT_Q,
        "H": qrcode.constants.ERROR_CORRECT_H
    }
    ec_level = level_map.get(error_level, qrcode.constants.ERROR_CORRECT_M)

    qr = qrcode.QRCode(
        version=None,
        error_correction=ec_level,
        box_size=8,
        border=4
    )
    qr.add_data(data_str)
    qr.make(fit=True)

    img = qr.make_image(fill_color="black", back_color="white").convert("RGB")
    return img, qr.version


# Chuyen anh PIL sang bytes
def image_to_bytes(pil_img):
    buf = io.BytesIO()
    pil_img.save(buf, format="PNG")
    return buf.getvalue()


# Doc noi dung QR code tu du lieu bytes cua anh
def decode_qr_image(file_bytes):
    file_arr = np.frombuffer(file_bytes, np.uint8)
    img = cv2.imdecode(file_arr, cv2.IMREAD_COLOR)

    if img is None:
        return None

    detector = cv2.QRCodeDetector()
    data, bbox, _ = detector.detectAndDecode(img)

    if not data:
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        data, bbox, _ = detector.detectAndDecode(gray)

    if not data:
        h, w = img.shape[:2]
        if w > 600:
            scale = 500.0 / w
            resized = cv2.resize(img, (int(w * scale), int(h * scale)))
            data, bbox, _ = detector.detectAndDecode(resized)

    return data if data else None


# Doc noi dung QR code truc tiep tu duong dan file anh
def decode_qr_file(file_path):
    if not os.path.exists(file_path):
        return None
    with open(file_path, "rb") as f:
        return decode_qr_image(f.read())
```